MARCO 3 · SEGURANÇA · 16/10/2026

Presença: como cada exigência do Marco 3 foi feita

Explicação completa, com o código de verdade, as saídas reais dos terminais, as telas do broker, diagramas e as medições. Cada seção começa com a anotação da aula e termina com a resposta.

Slides: `marco-03/apresentacao/Marco3-Slides.pdf`
Código: `projeto-presenca/marco-03/`

Software para Sistemas Ubíquos · UFG · Alany
Todos os trechos de código são copiados automaticamente dos arquivos (com o número da linha) e todas as saídas de terminal foram capturadas rodando o sistema.

Mapa: anotação da aula → onde está a resposta

O que foi pedido na aula	Situação	Seção
Principal: segurança	feito	1 (visão geral das camadas)
Canal criptografado	feito e testado	2
Autenticação MQTT	feito e testado	3
Todo dispositivo tem que falar quem é	feito e testado	4 e 5
Fazer atuação e autenticação	feito	6
Processo inverso: produtor do lado do consumidor e consumidor do lado do produtor	feito	6
Fazer funcionar redes de sensores	uma âncora real + N celulares; 3 âncoras no contrato	7
Verificar quantos dispositivos as âncoras conseguem monitorar ao mesmo tempo; se não, como vamos fazer	medido	8
Daria para fazer com Arduino? Prova de funcionalidade	sketch pronto, não gravado em placa	9
Validar: pelo ESP, a distância do sinal do celular	ferramenta pronta; medição real pendente	10
Validar: quantos dispositivos com o mesmo ESP32	ferramenta pronta (notebook e ESP32)	10

Antes de tudo: o que é o sistema

O **Presença** mede a **permanência** do estudante na sala ao longo da aula. Por pedido do professor no Marco 2, o **protótipo 1** virou o sistema oficial: o notebook faz o papel da **âncora** (um farol Bluetooth na parede), o **celular** do estudante mede a força do sinal (RSSI) e decide se está perto, e o **servidor** calcula a permanência e devolve o estado e os avisos. Tudo conversa por **MQTT**, através de um **broker**.

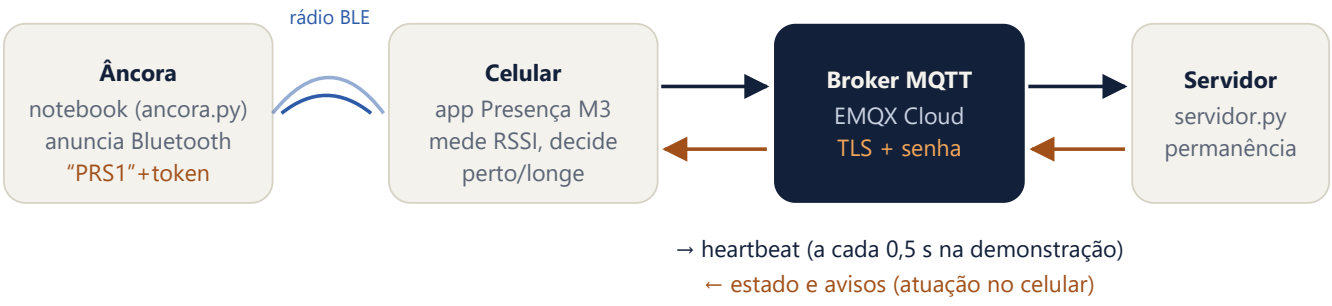


Figura 1 — As quatro peças. A âncora só transmite; quem mede é o celular; o broker leva as mensagens; o servidor decide e responde.

Onde fica cada coisa (pasta `marco-03/`, cópia do protótipo 1 — nada do Marco 2 foi alterado):

Pasta / arquivo	O que faz
`ancora/ancora.py`	o notebook vira âncora Bluetooth, com token HMAC
`ancora/escutar.py`	o “processo inverso” só para medir: o notebook ouve e conta aparelhos
`app/`	o app “Presença M3” (React Native/Expo), APK `presenca-m3.apk`
`servidor/servidor.py`, `servidor/seguranca.py`	servidor de permanência e toda a parte de segurança (sem rede, testável)
`contrato/v1/`	formato das mensagens (JSON Schema): `presenca.heartbeat.v2`, `seguranca.envelope.v1`
`ataques/ataques.py`, `ataques/carga.py`	o intruso (demonstração) e o teste de carga
`esp32-ancora/esp32-ancora.ino`	a mesma âncora em Arduino (ESP32)
`configurar.py`, `segredos.json`	gera e guarda os segredos (fora do git)

1 Segurança: a visão geral

> **Anotação:** “marco 03 segurança — principal: segurança”.

Segurança aqui responde a três perguntas, nesta ordem: **alguém consegue ler no caminho?** (canal cifrado), **alguém de fora consegue entrar?** (autenticação no broker) e **quem está dentro consegue se passar por outro?** (identidade de cada dispositivo). Uma quarta pergunta é específica deste sistema: **o estudante consegue dizer que está na sala estando em casa?** (prova de proximidade).

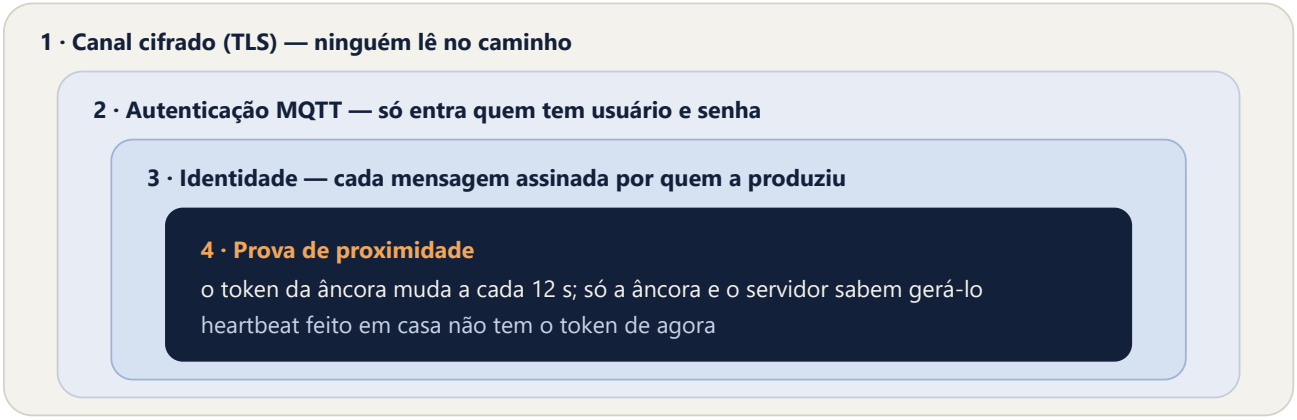


Figura 2 — As camadas, de fora para dentro. Cada uma cobre o que a anterior não cobre.

Camada	Impede	Como	Seção
1. TLS	ler ou alterar mensagens no Wi-Fi, no provedor, no caminho	broker só aceita portas cifradas (8883 e 8084)	2
2. Usuário e senha	qualquer pessoa conectar no broker	um usuário por papel no EMQX Cloud	3
3. Assinatura Ed25519	quem tem a senha falar em nome de outro (aluno, servidor, admin)	envelope assinado em toda mensagem	4
4. Token HMAC da âncora	“estou na sala” mandado de casa; repetir mensagem velha	token que só vale na janela de 12 s atual	5

> **Por que não basta senha no broker?** Porque a senha do papel “app” precisa ir **dentro do APK**: todo celular de estudante conecta com ela. Quem descompilar o APK a obtém. A senha separa “quem é do sistema” de “quem é de fora”, mas não separa um estudante do outro. Quem faz essa separação é a camada 3.

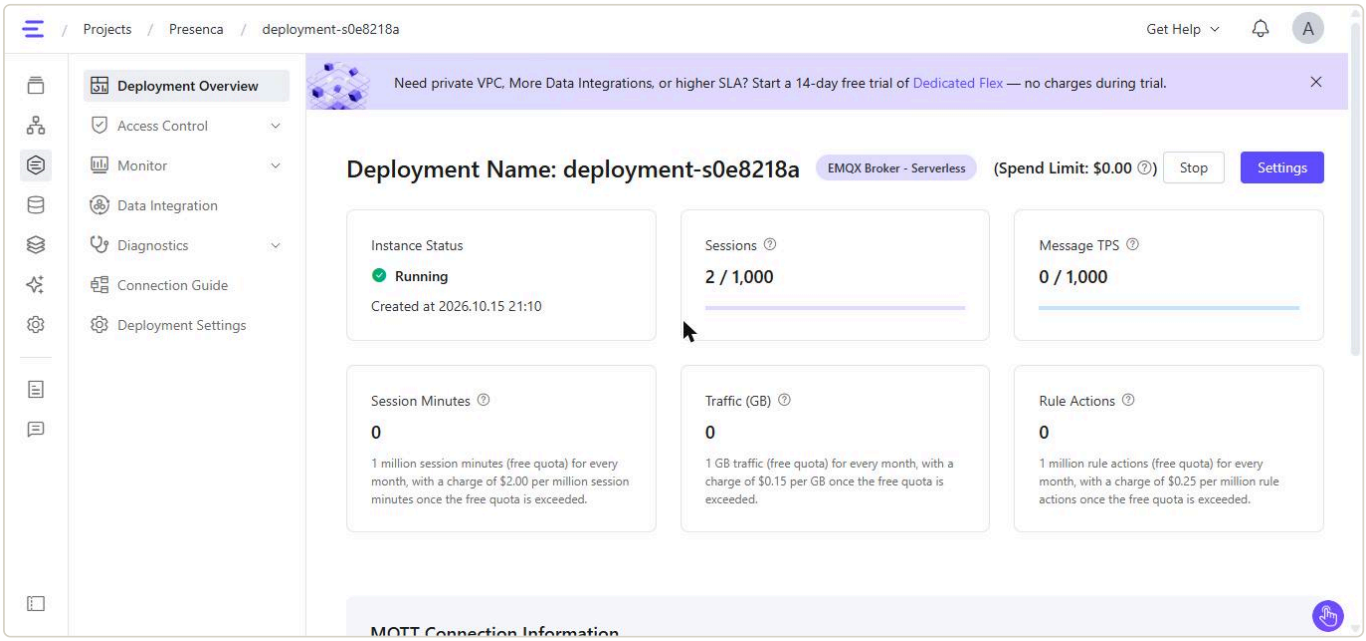
Por que EMQX e não HiveMQ

O professor indicou o HiveMQ. Na prática:

- O broker público `broker.hivemq.com`, usado até o Marco 2, **não tem autenticação nenhuma**: qualquer pessoa conecta, lê e publica (seção 3 mostra isso rodando).
- O **HiveMQ Cloud** deixou de ter plano gratuito para contas novas: o plano “Starter” pede cartão de crédito e custa cerca de US$ 249/mês depois de 15 dias; o “Enterprise” é por contato comercial (pede CNPJ).

- O **EMQX Cloud Serverless** tem cota gratuita mensal, **não pede cartão**, só aceita conexão com TLS e tem usuário/senha e ACL por tópico. Foi criado com limite de gasto de US$ 0,00 (“Free quota only”): se a cota acabar, ele suspende em vez de cobrar.

O protocolo é o mesmo (MQTT 3.1.1); trocar de broker mudou só o endereço e a porta, guardados em `segredos.json`.



Print 1 — Console do EMQX Cloud: deployment Serverless “Running”, limite de gasto US$ 0,00. As 2 sessões são o servidor e o celular conectados durante o ensaio.

2 Canal criptografado

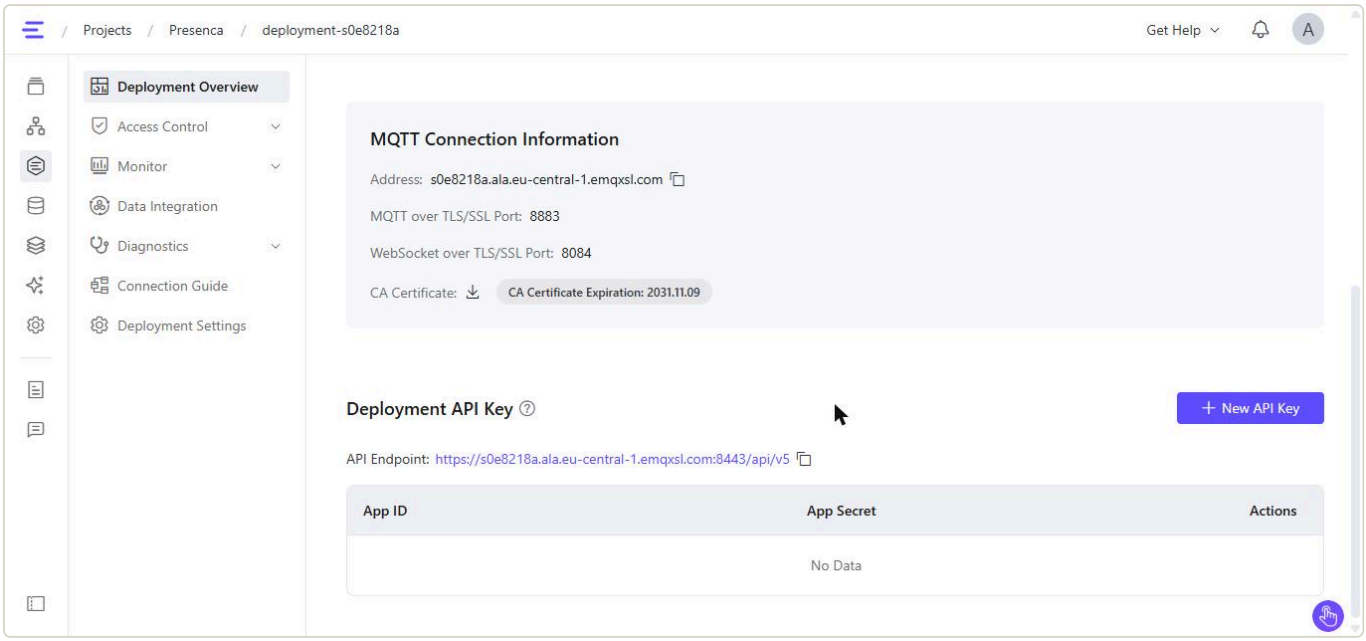
> **Anotação:** “canal criptografado”.

O que é

TLS (o mesmo cadeado do HTTPS) cifra tudo o que passa entre o aparelho e o broker e prova, por certificado, que o broker é mesmo o broker (e não alguém no meio fingindo ser). Sem TLS, qualquer um na mesma rede Wi-Fi lê o MQTT em texto puro — inclusive a senha, que vai na mensagem de conexão.

Como foi feito

No **broker**: o plano Serverless do EMQX só abre as portas cifradas — **8883** (MQTT sobre TLS) e **8084** (MQTT dentro de WebSocket sobre TLS, o “wss://”). As portas abertas 1883 e 8083 nem existem nesse plano.



Print 2 — Informação de conexão do broker: só as portas TLS (8883 e 8084), com certificado da autoridade válido até 2031.

No **servidor** (Python, paho-mqtt): `tls_set()` liga o TLS e confere o certificado do broker contra as autoridades do sistema. Não existe caminho sem TLS no código:

servidor/servidor.py linhas 111–131

```python
def criar_cliente(segredos, papel, client_id, websocket):
    """Cliente MQTT do papel ("servidor", "app") no EMQX Cloud.

    Nao existe caminho sem TLS: o EMQX Cloud so aceita conexao cifrada, e o
    certificado do broker e conferido (tls_set() usa as autoridades do
    sistema). Usuario e senha vem de segredos.json, um por papel.
    Devolve (cliente, porta).
    """
    transporte = "websockets" if websocket else "tcp"
    try:                                    # paho-mqtt 2.x
        cliente = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2,
                              client_id=client_id, transport=transporte)
    except (AttributeError, TypeError):     # paho-mqtt 1.x
        cliente = mqtt.Client(client_id=client_id, transport=transporte)
    cliente.tls_set()
    cred = segredos[papel]
    cliente.username_pw_set(cred["usuario"], cred["senha"])
    if websocket:
        cliente.ws_set_options(path=segredos.get("caminho_ws", "/mqtt"))
        return cliente, segredos.get("porta_wss", 8084)
    return cliente, segredos.get("porta_tls", 8883)
```

No **celular** (paho-mqtt em JavaScript): a conexão é `wss://…:8084/mqtt` com `useSSL: true`. O endereço vem de `segredos.ts`, gerado por `configurar.py` e fora do git:

app/src/lib/mqtt.ts linhas 37–40

```
    c.connect({
      useSSL: true,                // TLS: o HiveMQ Cloud não aceita conexão aberta
      userName: MQTT_USUARIO,      // autenticação MQTT do papel "app"
      password: MQTT_SENHA,
```

Prova

O ataque `senha` tenta primeiro a porta aberta 1883: a conexão nem abre.

Janela 3 — python ataques.py senha

```
== 2. EMQX Cloud — sem TLS, sem senha, senha errada ==
  atacante tenta: MQTT aberto, sem TLS, em s0e8218a.ala.eu-central-1.emqxsl.com:1883
  BLOQUEADO — a conexão nem abriu (TimeoutError). Ou o broker não atende nesta porta, ou a REDE bloqueia a porta 1883
(rede da faculdade: use o hotspot do celular)
  atacante tenta: TLS na porta 8883, mas SEM usuário e senha
  BLOQUEADO — o broker recusou: Bad user name or password
  atacante tenta: TLS com o usuário presenca-app e senha ERRADA
  BLOQUEADO — o broker recusou: Bad user name or password
  atacante tenta: TLS com usuário e senha certos (para comparar)
  CONECTOU — conectado
```

> **Resposta:** todo o tráfego (celular↔broker↔servidor) é cifrado com TLS e o certificado do broker é verificado. Não há porta sem cifra para usar, nem por engano.

3 Autenticação MQTT

> **Anotação:** “autenticação mqtt” · “fazer atuação e autenticação”.

O problema, rodando

No broker público usado até o Marco 2, o “intruso” conecta sem usuário e sem senha, assina todos os tópicos do projeto e ainda **publica** um heartbeat falso — o broker aceita e entrega:

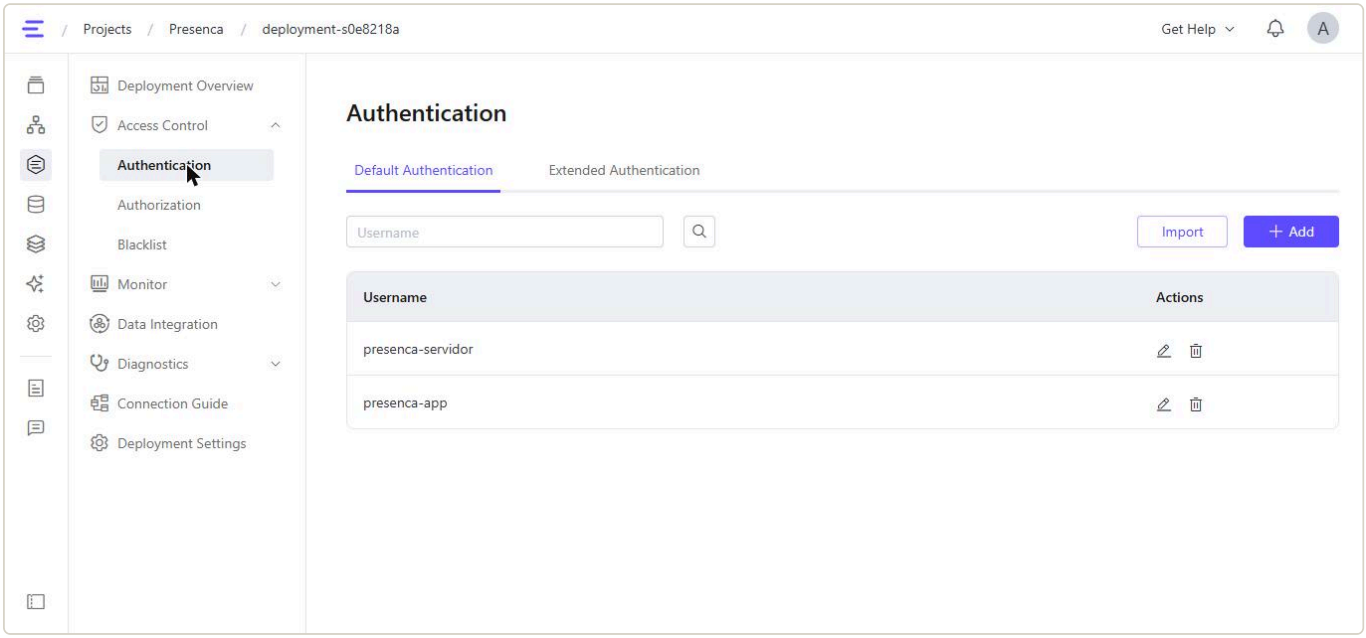
Janela 3 — python ataques.py publico --ws

```
== 1. Broker PÚBLICO (broker.hivemq.com) — como era até o Marco 2 ==
  atacante tenta: conectar em broker.hivemq.com:8884 SEM usuário e SEM senha
  CONSEGUIU — conectado
  atacante tenta: assinar ufg/ssu/presenca/# e PUBLICAR um heartbeat falso lá dentro, sem senha
  CONSEGUIU — publicou em ufg/ssu/presenca/alany/proto1/sala-pc/heartbeat e o broker entregou de volta: escrever e ler,
sem senha
  leu também 0 mensagem(ns) de terceiros nesses 8 s
  Qualquer pessoa no mundo pode ler e também PUBLICAR nesses tópicos.
```

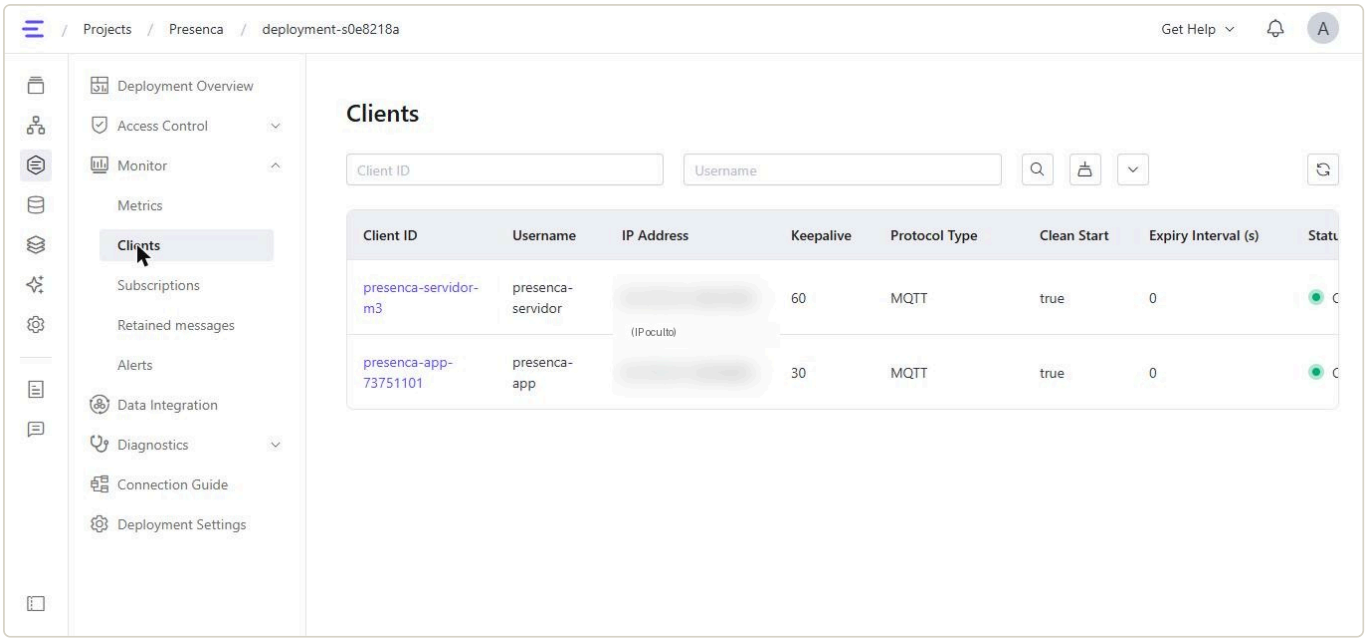
Como foi feito

No EMQX Cloud foram criados **dois usuários, um por papel** (Access Control → Authentication). Cada papel tem a sua senha forte (20 caracteres aleatórios gerados por `secrets` do Python), guardada só em `segredos.json`:

Usuário	Quem usa	Onde a senha fica
`presenca-servidor`	o servidor de permanência	`segredos.json` no notebook
`presenca-app`	todos os celulares (app)	`segredos.json` → `app/src/lib/segredos.ts` → dentro do APK



Print 3 — Os dois usuários cadastrados no broker (as senhas nunca aparecem no console).



Print 4 — Durante o ensaio: cada conexão aparece com o seu usuário. O servidor entra como presenca-servidor; o celular, como presenca-app.

No código, o usuário e a senha entram na conexão (`username_pw_set` no servidor, linhas acima; `userName/password` no app, seção 2). Se a senha estiver errada, o servidor avisa em vez de ficar tentando em silêncio:

servidor/servidor.py linhas 214–218

```
214     def _ao_conectar(self, cliente, userdata, flags, rc, properties=None):
215         if rc != 0:
216             print(f"{VERMELHO}{NEGRITO}O broker RECUSOU a conexão: {rc}. "
217                   f"Confira usuário e senha em segredos.json.{RESET}")
218             return
```

Prova

No mesmo ataque `senha` da seção anterior: com TLS mas **sem senha**, recusado; com o usuário certo e **senha errada**, recusado (“Bad user name or password”); com a senha certa, conecta. Testado também pela porta 8084 (WebSocket), com o mesmo resultado.

> **Resposta:** o broker só aceita quem apresenta usuário e senha válidos, por TLS. O intruso do broker público, que lia e escrevia tudo, agora não conecta.

> **Limite assumido:** a senha do app está no APK. Por isso a autenticação MQTT sozinha não basta — a seção 4 trata quem tem essa senha. O EMQX ainda permite restringir tópicos por usuário (ACL, em Access Control → Authorization); fica como próximo passo.

4 Todo dispositivo tem que falar quem é

> **Anotação:** “todo dispositivo tem que falar quem é”.

A ideia

Cada dispositivo tem um **par de chaves**: uma **privada**, que nunca sai dele, e uma **pública**, que vai junto de cada mensagem. Com a privada ele **assina**; com a pública qualquer um **confere** a assinatura. Mudou um byte da mensagem, a assinatura não confere. Sem a chave privada, não dá para produzir uma assinatura válida. Foi usado Ed25519 (RFC 8032): chaves de 32 bytes, assinaturas de 64 bytes, rápido e sem depender de número aleatório na hora de assinar.

O pseudônimo do estudante sai da chave

O nome que o servidor vê, `tok-xxxxxxxx`, é o começo do **SHA-256 da chave pública**. Consequência: não precisa de cadastro para provar que “tok-10ba682c” é dono de uma chave — basta recalcular o hash e conferir a assinatura. E ninguém consegue assinar como “tok-10ba682c” sem a chave privada que gerou esse nome. Nada de MAC, IMEI ou nome (decisão 5 do projeto).

app/src/lib/identidade.ts linhas 32–33

```
32 export async function minhaIdentidade(): Promise<Eu> {
33   if (cache) return cache;
```

app/src/lib/assinatura.ts linhas 41–43

```
41 export function identidadeDaChave(publicaHex: string) {
42   const h = bytesToHex(sha256(hexToBytes(publicaHex)));
43   return { aluno: `tok-${h.slice(0, 8)}`, deviceId: `cel-${h.slice(8, 12)}` };
```

O envelope assinado

Toda mensagem do sistema viaja dentro de um **envelope** (contrato `seguranca.envelope.v1`): a mensagem original vai intacta, como texto, no campo `carga`; ao lado vão quem diz ser (`de`), a chave pública e a assinatura. A assinatura cobre **o tópico MQTT + a carga**: uma mensagem válida para o aluno A não pode ser recolocada no tópico do aluno B, nem de outra sala.

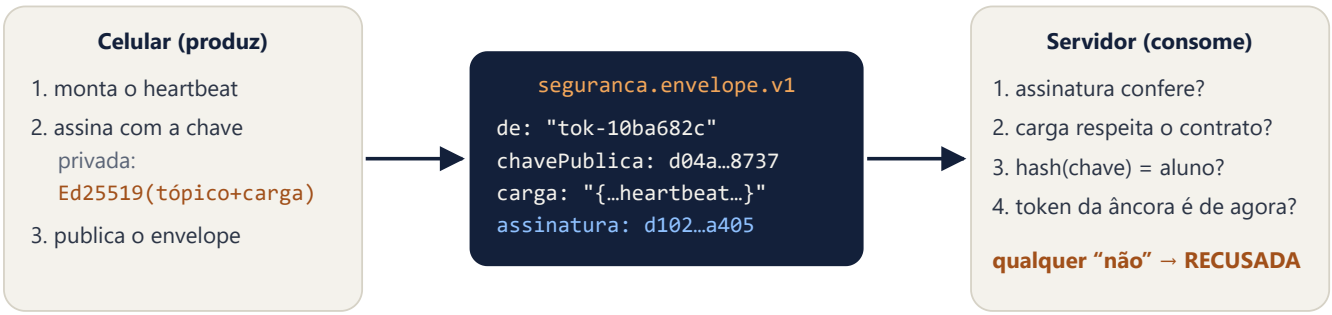


Figura 3 — O caminho de um heartbeat. O mesmo envelope é usado no sentido contrário: o servidor assina o estado e os avisos, e o celular confere.

Quem assina (no app, TypeScript, biblioteca noble-ed25519):

app/src/lib/assinatura.ts linhas 46–57

```ts
const mensagem = (topico: string, carga: string) => utf8ToBytes(topico + '\n' + carga);

export function envelopar(privadaHex: string, de: string, topico: string, carga: string | object): string {
  const texto = typeof carga === 'string' ? carga : JSON.stringify(carga);
  const env: Envelope = {
    eventType: TIPO_ENVELOPE,
    de,
    chavePublica: publicaDe(privadaHex),
    carga: texto,
    assinatura: bytesToHex(ed.sign(mensagem(topico, texto), hexToBytes(privadaHex))),
  };
  return JSON.stringify(env);
```

Quem confere (no servidor, Python, biblioteca cryptography). A ordem importa: primeiro a assinatura (quem é), depois o contrato, depois se o nome bate com a chave, por fim o token (onde estava):

servidor/servidor.py linhas 254–290

```python
    def _conferir(self, topico, payload):
        """Abre o envelope e confere quem mandou e se ouviu a ancora agora.

        Devolve o heartbeat (dict) ou levanta Recusa. A ordem importa: primeiro
        a assinatura (quem e), depois o contrato, depois o token (onde estava).
        """
        try:
            texto = payload.decode("utf-8")
        except UnicodeDecodeError:
            raise Recusa("não é texto")
        de, chave, dado = abrir_envelope(topico or self.cfg["topico_heartbeat"], texto)

        motivo = self._validar(dado)
        if motivo:
            raise Recusa("FORA DO CONTRATO", motivo)

        aluno, device = identidade_da_chave(chave)
        if dado["aluno"] != aluno or dado["deviceId"] != device or de != aluno:
            raise Recusa("IDENTIDADE TROCADA",
                         f"assinou com a chave de {aluno}, mas diz ser {dado['aluno']}")

        atras = ATRASO_FILA_JANELAS if dado.get("bufferizado") else JANELAS_AO_VIVO
        for a in dado["ancoras"]:
            if a["rssi"] is None:
                continue                        # nao ouviu: nao ha o que provar
            segredo = self.segredos.get("ancoras", {}).get(a["id"])
            if segredo is None:
                raise Recusa("ÂNCORA DESCONHECIDA", f"{a['id']} não é desta sala")
            if not token_valido(segredo, a["id"], a.get("token"), atras=atras):
                raise Recusa("TOKEN VENCIDO",
                             f"o token {a.get('token')} não é o que a {a['id']} anuncia agora "
                             f"(heartbeat inventado ou requentado)")

        if aluno not in self.chaves_vistas:
            self.chaves_vistas[aluno] = chave
            self._aviso_linha(CIANO, "+ APARELHO", f"{aluno} se identificou (chave {chave[:16]}…)")
        return dado
```

Os quatro dispositivos e como cada um prova quem é

Dispositivo	Prova	Quem confere	Se falhar
Celular do estudante	assina cada heartbeat; o nome `tok-…` é o hash da chave	servidor	`SEM ASSINATURA`, `ASSINATURA FALSA`, `IDENTIDADE TROCADA`
Âncora	anuncia um token HMAC que só ela e o servidor sabem gerar (seção 5)	servidor (o celular só repassa)	`TOKEN VENCIDO`, `ÂNCORA DESCONHECIDA`
Servidor	assina estado e avisos; a chave pública dele vem fixada no app	celular	o celular ignora e conta como "mensagem falsa"
Celular do administrador	assina a turma (grade e limiar de 1 m); o estudante guarda essa chave no pareamento (código/QR, uma vez)	celular do estudante	"Turma recusada: assinada por outro administrador"

Prova

Com a senha do app (como se tivesse vazado do APK), o intruso tenta três coisas. Do lado dele:

Janela 3 — python ataques.py forjar

```
== 3. Senha do app VAZADA — tentar falar em nome de outro aluno ==
  A senha do papel 'app' está dentro do APK: quem descompilar consegue.
  Por isso a segurança não pode parar no broker.
  (escutando ufg/ssu/presenca/alany/m3/sala-pc/heartbeat por até 15 s para achar um aluno…)
  nenhum heartbeat no ar; usando um pseudônimo inventado: tok-12345678
  atacante tenta: a) heartbeat SEM assinatura dizendo que tok-12345678 está na sala
  onde olhar: terminal do servidor → x RECUSADA  SEM ASSINATURA
  atacante tenta: b) heartbeat ASSINADO com a chave do atacante, mas dizendo ser tok-12345678
  onde olhar: servidor → x RECUSADA  IDENTIDADE TROCADA (a chave é de tok-bf200741, não de tok-12345678)
  atacante tenta: c) heartbeat assinado como ele mesmo (tok-bf200741), de casa, com token inventado
  onde olhar: servidor → x RECUSADA  TOKEN VENCIDO (não ouviu a âncora)
```

Do lado do servidor, conectado ao EMQX real, as três foram recusadas, cada uma pelo motivo certo:

Janela 2 — python servidor.py

```
== Presença · servidor de permanência ==
Teclas: n = aula nova · p = resumo · q = sair (sempre com Enter)
Conectado ao s0e8218a.ala.eu-central-1.emqxsl.com, porta 8084 (MQTT sobre WebSocket com TLS, usuário presenca-servidor).
Chave pública do servidor (o app confere o estado e os avisos com ela):
  da7ed666d373155cc3350fcc6b21c659f0a93abf15735874f4461d44c2d0b4df
Esperando as mensagens do celular do estudante.

── AULA s557b COMEÇOU ──────────────────────────
  dura 100 s · pode sair por até 30 s no total · avisos aos 20 s e 26 s fora
  (na demonstração, 1 s = 30 s de uma aula de verdade)

   tempo  mensagem  onde          situação                tempo que contou
  x RECUSADA      SEM ASSINATURA — mensagem fora do envelope assinado
  x RECUSADA      IDENTIDADE TROCADA — assinou com a chave de tok-4481657a, mas diz ser tok-12345678
  x RECUSADA      TOKEN VENCIDO — o token 1a2b3c4d não é o que a anc-pc-1 anuncia agora (heartbeat inventado ou
requentado)
  … SILÊNCIO      nada chega há 6 s (rede, bateria ou app); nada é decidido sem mensagem
```

Os mesmos casos (e outros: carga alterada depois de assinada, assinatura de outro tópico) são testes automáticos:

```
python -m pytest testes -v -k "...segurança..."

test_heartbeat_de_exemplo_respeita_o_contrato PASSED [ 15%]
test_respostas_respeitam_os_contratos PASSED    [ 23%]
test_campo_desconhecido_e_rejeitado_pelo_contrato PASSED [ 30%]
test_mensagem_sem_assinatura_e_recusada PASSED  [ 38%]
test_carga_alterada_depois_de_assinada_e_recusada PASSED [ 46%]
test_assinatura_de_outro_topico_e_recusada PASSED [ 53%]
test_falar_em_nome_de_outro_aluno_e_recusado PASSED [ 61%]
test_token_de_outra_janela_e_recusado PASSED    [ 69%]
test_token_inventado_e_recusado PASSED          [ 76%]
test_heartbeat_da_fila_aceita_token_de_ate_48_s PASSED [ 84%]
test_ancora_nao_ouvida_nao_precisa_de_token PASSED [ 92%]
test_respostas_saem_assinadas_pelo_servidor PASSED [100%]
====================== 13 passed, 36 deselected in 0.33s ======================
```

> **Resposta:** cada mensagem diz quem a produziu e prova com uma assinatura que só aquele dispositivo consegue fazer. O servidor recusa o que não prova; o celular recusa estado, aviso e turma que não venham de quem deveria.

5 A âncora também diz quem é: o token HMAC

> **Anotação:** “todo dispositivo tem que falar quem é” — inclusive a âncora, que é o dispositivo que prova onde o estudante está.

A assinatura da seção 4 prova **quem** mandou, mas não **de onde**. Um estudante com o app modificado poderia assinar com a própria chave um heartbeat “estou perto” estando em casa. A âncora fecha essa porta.

Como foi feito

A âncora anuncia por Bluetooth: `"PRS1" | nº da âncora | token (4 bytes)`. O token deixou de ser sorteado: agora é

`token = HMAC-SHA256( segredo da âncora , "anc-pc-1|" + janela )[0..3]`, com `janela = ⌊ hora atual ÷ 12 s ⌋`

HMAC (RFC 2104) é um “carimbo” que só quem tem o segredo consegue produzir. O segredo fica só na âncora e no servidor (`segredos.json`, gerado por `configurar.py`). Como a janela segue o relógio, a âncora e o servidor calculam **o mesmo token sem conversar**. O celular não tem o segredo: só **repassa** no heartbeat o token que ouviu (campo novo `ancoras[].token` do contrato `presenca.heartbeat.v2`).

servidor/seguranca.py linhas 158–173

```
158 def token_da_ancora(segredo_hex, ancora, num_janela):
159     """4 bytes de HMAC-SHA256(segredo, "anc-pc-1|<janela>"), em hex.
160
161     E o que ancora.py anuncia por Bluetooth naquela janela de 12 s.
162     """
163     msg = f"{ancora}|{num_janela}".encode("utf-8")
164     return hmac.new(bytes.fromhex(segredo_hex), msg, hashlib.sha256).digest()[:TOKEN_BYTES].hex()
165
166
167 def token_valido(segredo_hex, ancora, token, agora_s=None, atras=JANELAS_AO_VIVO):
168     """O token e de uma janela entre `atras` janelas atras e a proxima?"""
169     if not isinstance(token, str):
170         return False
171     j = janela(agora_s)
172     return any(hmac.compare_digest(token_da_ancora(segredo_hex, ancora, j + d), token)
173                for d in range(-atras, 2))
```

Na âncora (notebook), o token é trocado exatamente na virada de cada janela:

ancora/ancora.py linhas 104–112

```
104             # A janela segue o relogio (como no servidor), nao o momento em que
105             # a ancora ligou: os dois calculam o mesmo token sem conversar.
106             j = janela()
107             token = token_da_ancora(segredo, NOME, j)
108             if pub is not None:
109                 pub.stop()
110             pub = novo_publicador(token)
111             pub.start()
112             print(f"{time.strftime('%H:%M:%S')}  janela {j}  token {token}")
```

Janela 1 — python ancora.py (cada linha, uma janela de 12 s com o seu token)

```
== Presença · Marco 3 · âncora (este notebook) ==
Anunciando por Bluetooth como anc-pc-1. Token novo a cada 12 s, assinado com o segredo da âncora (HMAC).
Deixe esta janela aberta. Ctrl+C para parar.

02:41:05  janela 149344105  token 45cb0d0e
           rádio: criado
           rádio: TRANSMITINDO
02:41:12  janela 149344106  token 42f6d47b
           rádio: criado
           rádio: TRANSMITINDO
02:41:24  janela 149344107  token bf18fee7
           rádio: criado
           rádio: TRANSMITINDO
```

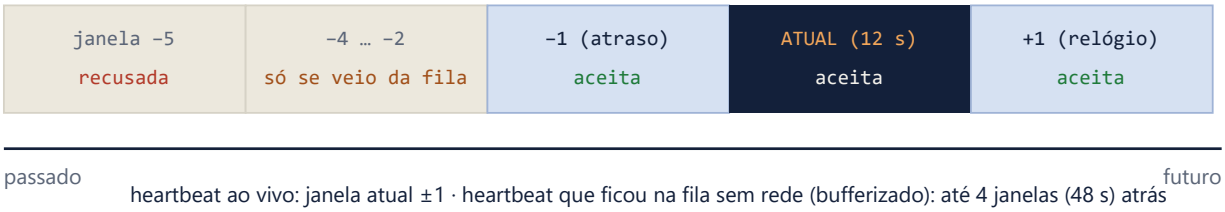


Figura 4 — Quais tokens o servidor aceita. A margem cobre atraso de rede e relógio um pouco adiantado; a fila de 30 s do app cabe em 48 s.

> **Resposta:** a âncora prova que é ela por um token que só ela (e o servidor) sabe gerar, e que vale por segundos. Heartbeat inventado em casa não tem o token de agora; heartbeat antigo repetido vence.

> **Limite assumido (já previsto no projeto):** alguém dentro da sala retransmitindo o token ao vivo para quem está em casa (“relay”) continua possível. O token tem 4 bytes. Âncora e servidor precisam de relógio certo (aqui são o mesmo notebook; no ESP32, NTP).

6 Atuação e o processo inverso (produtor ↔ consumidor)

> **Anotação:** “fazer atuação e autenticação” · “processo inverso: produtor do lado do consumidor e consumidor do lado do produtor”.

O que significa aqui

Num fluxo simples de IoT, o sensor só produz e o servidor só consome. No Presença os dois lados fazem os dois papéis: o **celular** produz o heartbeat **e** consome o estado e os avisos; o **servidor** consome o heartbeat **e** produz o estado e os avisos. A resposta do servidor causa uma **atuação** no celular — vibração, notificação e o cartão na tela (“notificar antes de punir”). É isso que fecha o laço ciber-físico.

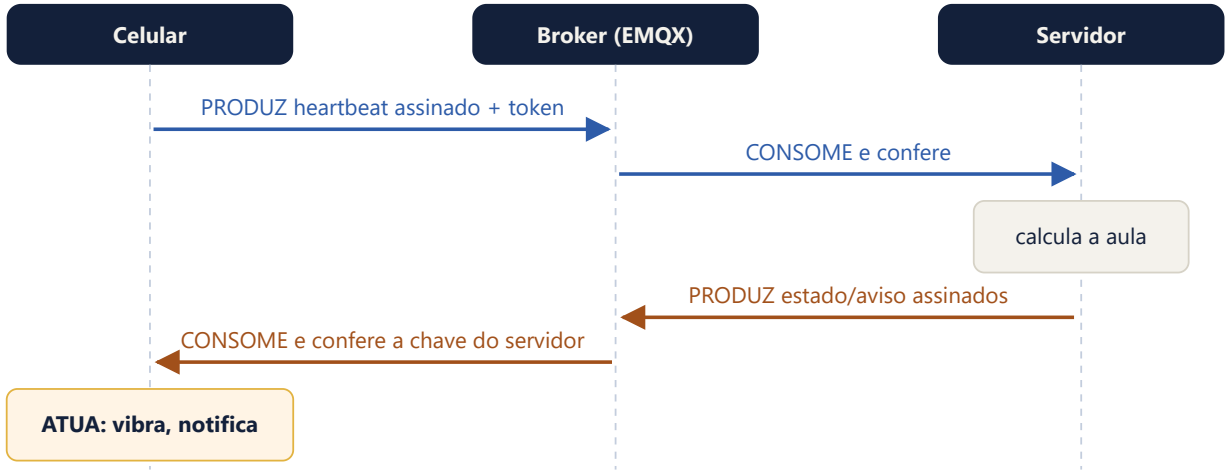


Figura 5 — Os dois lados produzem e consomem. Os dois assinam o que produzem e conferem o que consomem.

Como foi feito

O servidor assina cada estado antes de publicar (publicado como mensagem retida, por aluno):

servidor/servidor.py linhas 424–425

```
424         topico = f"{self.cfg['topico_estado']}/{aluno}"
425         self.cliente.publish(topico, envelopar(self.chave, "servidor", topico, msg), qos=0, retain=True)
```

O celular só aceita o que o **servidor** assinou — a chave pública do servidor é fixada no app por `configurar.py`. E o heartbeat sai assinado com a chave do celular:

app/src/lib/usar-aparelho.ts linhas 73–78

```
73      // Só vale o que o SERVIDOR assinou (chave fixada no app por configurar.py).
74      // Quem tem a senha do broker consegue publicar aqui, mas não consegue
75      // forjar "PRESENTE, 100 %" em nome do servidor.
76      const pararEstado = mqtt.assinar(topicos.estado(sala, eu.aluno), (topico, texto) => {
77        const a = abrirDe(topico, texto, CHAVE_PUBLICA_SERVIDOR);
78        if (!a.ok) return recusar(`estado: ${a.motivo}`);
```

app/src/lib/usar-aparelho.ts linhas 104–106

```
104         // Todo heartbeat sai assinado pela chave do aparelho (envelope).
105         (json) => mqtt.publicar(topicos.heartbeat(sala),
106           envelopar(eu.privada, eu.aluno, topicos.heartbeat(sala), json)),
```

A atuação, quando chega um aviso válido:

app/src/app/turma/[codigo].tsx linhas 79–85

```
79   const aoAviso = useCallback(async (a: AvisoServidor) => {
80     if (!(await registrarAviso(db, codigo, a))) return;      // já mostrado: não toca de novo
81     setAvisoNaTela(a.texto);
82     Vibration.vibrate([0, 400, 200, 400]);
83     notificar(a.tipo === 'FIM_DA_AULA' ? 'Fim da aula' : 'Presença', a.texto);
84     setAvisos(await ultimosAvisos(db, codigo));
85   }, [db, codigo]);
```

A turma (que traz o limiar de 1 m) também é produzida por um celular — o do administrador — e consumida pelo do estudante; só é aceita se vier assinada pela chave guardada no pareamento:

app/src/lib/rede-turma.ts linhas 63–69

```
63 export function acompanharTurma(codigo: string, aoMudar: (t: Turma) => void, aoRecusar?: (motivo: string) =>
void) {
64   return mqtt.assinar(topicos.turma(codigo), async (topico, texto) => {
65     const r = abrir(codigo, topico, texto);
66     if (!r) return aoRecusar?.('turma sem assinatura válida');
67     const fixada = await SecureStore.getItemAsync(chaveDoAdmin(codigo));
68     if (fixada && fixada !== r.chave) return aoRecusar?.('turma assinada por outro administrador');
69     aoMudar(r.turma);
```

Prova

`python ataques.py estado` publica no tópico do estudante um estado falso “PRESENTE, 100 %”, sem assinatura e com assinatura de outra chave; o cartão **Segurança** do app passa a mostrar “Recusei 2 mensagem(ns) falsa(s)” e a tela não muda. `python ataques.py turma` publica uma turma com limiar −100 dBm (que faria todo mundo parecer perto, até de casa); o app mostra “Turma recusada: turma assinada por outro administrador”.

> **Resposta:** o celular é produtor e consumidor; o servidor também. A atuação acontece no celular e só é disparada por mensagem comprovadamente do servidor.

7 Rede de sensores funcionando

> **Anotação:** “fazer funcionar redes de sensores”.

No Presença, a “rede de sensores” tem dois tipos de nó: as **âncoras** (faróis fixos na parede) e os **celulares**, que são os sensores de verdade — cada um mede o RSSI das âncoras com o próprio rádio. O servidor junta as medições de todos e decide a permanência de cada estudante, com uma sessão separada por estudante.

Nó	No projeto (sala real)	No Marco 3 (funcionando)
Âncoras	3 ESP32 por sala, trilateração + polígono	**1 âncora real** (o notebook); o ESP32 em Arduino pronto (seção 9)
Celulares	todos os estudantes	o celular real + até centenas simulados pelo `carga.py`, cada um com a sua chave
Broker	—	EMQX Cloud, TLS + senha
Servidor	permanência por estudante	idem, conferindo assinatura e token de cada nó

O contrato já aceita até 3 âncoras por heartbeat, cada uma com o seu token (`ancoras[]`), e o servidor confere o token de **cada** âncora ouvida contra o segredo daquela âncora — acrescentar âncoras é acrescentar segredos em `segredos.json`. No celular, o token de cada anúncio ouvido entra no heartbeat:

app/src/lib/aparelho.ts linhas 206–208

```
206      ancoras: id.ancoras.map((a, i) => (r.rssi[i] === null
207        ? { id: a, rssi: null, token: null }
208        : { id: a, rssi: um(r.rssi[i]!), token: r.tokens[i] ?? null })),
```

> **Resposta:** a rede funciona ponta a ponta com uma âncora real, o celular real e celulares simulados; cada nó se identifica (seções 4 e 5). **O que não está feito:** três âncoras físicas e a malha ESP-NOW entre elas (previstas no projeto).

8 Quantos dispositivos as âncoras monitoram ao mesmo tempo?

> **Anotação:** “verificar quantos dispositivos essas âncoras conseguem monitorar ao mesmo tempo, se não como vamos fazer”.

A âncora não monitora ninguém: é um farol

Pela decisão 3 do projeto, a âncora **só transmite**. Um anúncio Bluetooth é ouvido por todos os celulares em volta ao mesmo tempo — como uma lâmpada é vista por todos na sala. Quem mede é cada celular, com o próprio rádio. **Não existe limite de celulares por âncora.** O limite, se houver, está no caminho de volta: broker e servidor. Esse caminho foi medido.

Medição 1 — o servidor sozinho

Sem rede, 200 celulares simulados, cada mensagem passando por tudo (abre envelope, confere assinatura e token, recalcula a aula, assina a resposta):

python carga.py 200 --tempo 20 --sem-rede

```
== Servidor sem rede: 200 celulares × 40 heartbeats (20 s de aula) ==
  aceitos: 8000 de 8000 · recusados: 0
  por mensagem (abre envelope, confere assinatura e token, recalcula a aula, assina a resposta):
     média 0.42 ms · 95 % abaixo de 0.62 ms
  ≈ 2405 mensagens por segundo num núcleo deste notebook
  com heartbeat a cada 60 s (aula real): ≈ 144328 celulares
```

Medição 2 — com o broker de verdade

python carga.py 5 --tempo 15

```
$ python servidor.py --quieto --tls          (outra janela)
$ python carga.py 5 --tempo 15
Conectando 1 cliente(s) MQTT ao EMQX Cloud…
Simulando 5 celulares, 1 heartbeat a cada 0.5 s, por 15 s…

== Resultado: 5 celulares ==
  heartbeats enviados:  155  (10 por segundo)
  estados de volta:     155  (100.0 %)
  ida e volta (celular → broker → servidor → broker → celular):
     mediana 502 ms · 95 % abaixo de 1156 ms · pior 1297 ms
```

python carga.py 30 --tempo 15

```
$ python carga.py 30 --tempo 15
Simulando 30 celulares, 1 heartbeat a cada 0.5 s, por 15 s…

== Resultado: 30 celulares ==
  heartbeats enviados:  930  (62 por segundo)
  estados de volta:     177  (19.0 %)
  ida e volta (celular → broker → servidor → broker → celular):
     mediana 7497 ms · 95 % abaixo de 14234 ms · pior 15114 ms
```

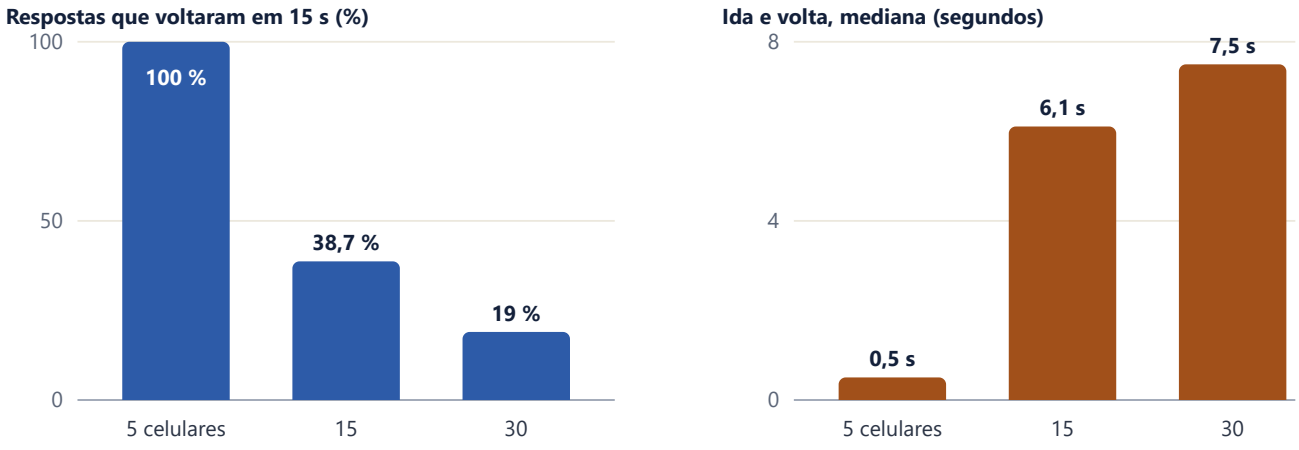


Figura 6 — Teste de carga com o broker EMQX real, heartbeat a cada 0,5 s (ritmo comprimido da demonstração). Até 5 celulares, tudo volta em meio segundo; acima disso forma fila.

Onde está o gargalo — medição 3

O servidor faz ~2.400 mensagens/s, então a fila não vem dele. Uma sonda sem servidor no meio — um cliente publica 60 mensagens/s, outro conta quantas chegam — mostrou que **o próprio EMQX gratuito deixa passar ~11 mensagens por segundo por conexão**:

Sonda do broker

```
$ python sonda.py      (um cliente publica 60 msg/s, outro conta; sem servidor no meio)
enviadas 600 (60/s) · recebidas 157 · nos 10 s: 10.7/s
```

O servidor usa **uma** conexão para receber e responder tudo, então é nele que o limite aparece. Na aula real o heartbeat é a cada **60 s** (não 0,5 s): 11 mensagens/s × 60 s ≈ **650 celulares por conexão do servidor**.

Medida	Resultado
Celulares por âncora	**sem limite** (farol)
Servidor, sem rede	0,4 ms por mensagem ≈ 2.400/s
EMQX Serverless gratuito	~11 mensagens/s por conexão; 1.000 sessões; 1 GB/mês
Aula real (heartbeat de 60 s), 1 conexão do servidor	≈ 650 celulares

> **Resposta — "se não, como vamos fazer":** a âncora não limita. Para passar de ~650 celulares, o servidor ganha cópias, cada uma com a sua conexão, dividindo os heartbeats por assinatura compartilhada do MQTT (`$share/grupo/…/heartbeat`): o broker entrega cada mensagem a uma só cópia. Outra saída é o plano dedicado do broker, sem esse limite por conexão.

9 Daria para fazer com Arduino? Prova de funcionalidade

> **Anotação:** “daria para fazer o que eu quero com arduino — prova de funcionalidade”.

Sim. `esp32-ancora/esp32-ancora.ino` é a **mesma âncora** em Arduino para ESP32 (placa “ESP32 Dev Module”, núcleo Espressif 3.x): anuncia o mesmo formato `PRS1 | nº | token`, com o **mesmo token HMAC** da janela de 12 s, e pega a hora certa por NTP no Wi-Fi. O app e o servidor não mudam nada para aceitá-la. Usa só o que já vem no núcleo do ESP32: `BLEDevice` para o Bluetooth e `mbedtls` para o HMAC.

esp32-ancora/esp32-ancora.ino linhas 52–63

```
52 void tokenDaJanela(uint32_t janela, uint8_t token[4]) {
53   char msg[48];
54   snprintf(msg, sizeof msg, "%s|%lu", NOME_ANCORA, (unsigned long)janela);
55   uint8_t h[32];
56   mbedtls_md_context_t ctx;
57   mbedtls_md_init(&ctx);
58   mbedtls_md_setup(&ctx, mbedtls_md_info_from_type(MBEDTLS_MD_SHA256), 1);
59   mbedtls_md_hmac_starts(&ctx, segredo, sizeof segredo);
60   mbedtls_md_hmac_update(&ctx, (const uint8_t*)msg, strlen(msg));
61   mbedtls_md_hmac_finish(&ctx, h);
62   mbedtls_md_free(&ctx);
63   memcpy(token, h, 4);
```

esp32-ancora/esp32-ancora.ino linhas 133–140

```
133 void loop() {
134   if (MODO_ESCUTA) { escutar(); return; }
135   uint32_t janela = (uint32_t)(time(nullptr) / TOKEN_S);
136   if (janela != janelaAtual) {            // a janela segue o relógio, como no servidor
137     janelaAtual = janela;
138     anunciar(janela);
139   }
140   delay(100);
```

O mesmo sketch tem um **modo escuta** (`MODO_ESCUTA = true`): em vez de anunciar, varre os anúncios em volta, conta quantos aparelhos distintos ouve e mostra o RSSI de cada um — usado na seção 10.

> **Honestidade:** o sketch **não foi gravado em placa** — não havia ESP32 em mãos. Nos testes, o notebook faz o papel do ESP32 com exatamente o mesmo anúncio e o mesmo token. A âncora de parede custaria cerca de R$ 30 e fica na tomada.

10 Validações: distância do celular e quantos aparelhos por ESP32

> **Anotação (validações):** "1. verificar pelo ESP a distância do sinal do celular" · "quantos dispositivos com o mesmo ESP32".

A ferramenta: o processo inverso, só para medir

No sistema, a âncora não escuta (decisão 3). Para **medir**, existe o caminho inverso: `ancora/escutar.py` (no notebook) e o modo escuta do ESP32 varrem os anúncios Bluetooth em volta e, a cada 3 s, mostram **quantos aparelhos distintos** o rádio ouviu, o RSSI de cada um e a **distância estimada**.

ancora/escutar.py linhas 51–53

```
51 def distancia(rssi, rssi_1m, n):
52     """Modelo log-distancia: d = 10 ^ ((RSSI@1m - RSSI) / (10 n))."""
53     return 10 ** ((rssi_1m - rssi) / (10 * n))
```

Para o celular aparecer, ele precisa **anunciar** (o app Presença não anuncia, por privacidade): no Android, o app gratuito *nRF Connect* → Advertiser cria um anúncio com nome. Depois: `python escutar.py --alvo "nome"`.

Distância a partir do sinal

Modelo log-distância, o mesmo usado na tela do app:

$$RSSI(d) = RSSI(1\ m) - 10 \cdot n \cdot \log_{10}(d) \Rightarrow d = 10^{((RSSI(1\ m) - RSSI) / (10 \cdot n))}$$

O **RSSI a 1 m** é medido de verdade (calibração do administrador, 8 s parado a 1 m, mediana). O expoente **n = 2,2** é estimado (típico de ambiente interno). Só a distância *mostrada* depende de n; a decisão perto/longe vem direto do limiar calibrado.

Próximo passo (medição real): afastar-se 0,5 / 1 / 2 / 3 / 4 / 5 m do notebook, anotar o dBm em `medicoes/distancia.csv` e rodar `python medicoes/ajustar_distancia.py`, que ajusta n por mínimos quadrados e mostra a faixa real em que "1 m" cai.

> **Resposta:** as duas validações têm ferramenta pronta (notebook e ESP32) e o método está definido. **Os números reais dependem do teste presencial** — afastando-se do notebook e com vários celulares anunciando.

> **Limite:** Android e iPhone trocam o endereço Bluetooth de tempos em tempos, por privacidade. A contagem é de *anunciantes vistos na janela*, não de pessoas. RSSI não é régua: mão, bolso e corpo mudam 5 a 10 dB.

11 Testes automáticos, limites e como reproduzir

Testes

python -m pytest testes -q (servidor)

```
.............................................                [100%]
49 passed in 0.79s
```

O app tem 22 testes, entre eles a prova de que o JavaScript e o Python calculam **o mesmo pseudônimo** para a mesma chave e de que mensagem alterada, de outro tópico ou com chave que não é a do servidor é recusada (`app/src/lib/assinatura.test.ts`).

Limites assumidos

Limite	Por quê / mitigação
Chave do celular no armazenamento seguro, não dentro do chip	o Keystore não oferece Ed25519 pelo Expo; celular com root a extrai
Senha do app no APK	por isso existe a assinatura; ACL por tópico no EMQX é o próximo passo
Relay do token em tempo real	residual, já previsto no projeto
Primeiro pareamento confia na turma que chegar	é o momento do QR, uma vez por semestre
~11 mensagens/s por conexão no EMQX gratuito	≈ 650 celulares por conexão na aula real; assinatura compartilhada para mais
ESP32 não gravado; 1 âncora física	notebook faz o papel; sketch pronto

Como reproduzir

Janela	Comando (a partir de `projeto-presenca\marco-03`)
uma vez	`python configurar.py` e `powershell -ExecutionPolicy Bypass -File .\gerar-apk.ps1`
1 · âncora	`cd ancora` → `python ancora.py`
2 · servidor	`cd servidor` → `python servidor.py`
celular	Presença M3 → admin publica, calibra a 1 m, inicia a aula → aluno entra pelo código
3 · ataques	`cd ataques` → `python ataques.py senha | forjar | estado | turma | publico --ws`
carga	`python carga.py 5` (servidor com `--quieto --tls`) · `python carga.py 200 --sem-rede`

Referências

- MQTT 3.1.1 (OASIS): docs.oasis-open.org/mqtt/mqtt/v3.1.1
- EMQX Cloud (documentação): docs.emqx.com/en/cloud/latest
- Assinatura compartilhada no EMQX: docs.emqx.com/.../mqtt-shared-subscription

- Broker público da HiveMQ: hivemq.com/mqtt/public-mqtt-broker
- Ed25519 — RFC 8032: rfc-editor.org/rfc/rfc8032
- HMAC — RFC 2104: rfc-editor.org/rfc/rfc2104
- noble-ed25519 (app): github.com/paulmillr/noble-ed25519 · cryptography (servidor): cryptography.io
- Modelo log-distância: en.wikipedia.org/wiki/Log-distance_path_loss_model
- Slides do Marco 3: `marco-03/apresentacao/Marco3-Slides.pdf`